

Configurando el entorno de trabajo en Visual Studio

Con este panorama, vamos a trabajar con el código. Aquí tenemos nuestro repositorio, en realidad nuestro Visual Studio, que es un repositorio que en la clase o actividad anterior finalizamos con un `git init`. Les invitamos a que revisen esa actividad y observen el color del código antes del `git init`. Lo verán en blanco. Ahora, si lo observan, está en verde y tiene una letra "U". Esa letra "U" indica un *tracker*. ¿Qué significa esto? Que Git sabe que hay una caja, pero que nada de ese código está siendo seguido o gestionado por Git. Eso fue lo que hicimos con `git init`, creamos la caja. Visual Studio muestra los colores, y estos nos guiarán bastante bien.

Lo siguiente que haremos es seguir la receta que está en el repositorio que creamos en la nube. Si revisan, hay una receta que nos indica los pasos. Vamos a seguir esos pasos. Primero, utilizamos el comando `git add` para agregar archivos al área de preparación. En lugar de agregar uno a uno, porque tenemos siete archivos, agregaremos el directorio actual. Entonces, ejecutamos:

```
git add .
```

Con esto, logramos subir los elementos. Lo que hicimos fue agregar a la caja local los elementos, y observen lo que sucede aquí. El color verde cambió a un verde tenue, la "U" desapareció y aparece "added", indicando que fue agregado a la caja.

Realizando configuraciones de usuario y commit

Luego, debemos hacer un *commit*, que es como tomar una foto. Todo esto lo veremos en detalle más adelante, no se preocupen. Para hacer esa foto, debemos configurar nuestro usuario y correo electrónico. Para ello, ejecutamos:

```
git config --global user.email
```

Verificamos que sea correcto: `leocastillo.aluralatam@gmail.com`. Aquí cometimos un error, disculpen. Es `git config`, sin comillas, o si lleva comillas, debemos cerrarlas. Así que configuramos correctamente con:

```
git config --global user.email "leocastillo.aluralatam@gmail.com"
```

Perfecto. Ahora configuramos nuestro nombre público con:

```
git config --global user.name "Leonardo Castillo Lacruz"
```

Esto se hace solo una vez. Para verificar si todo está bien configurado, ejecutamos:

```
git config --global --list
```

Perfecto, nuestros datos están correctamente configurados. Ahora podemos tomar la foto con `git commit`. Observen cómo cambia el color del código. El *commit* lleva un nombre, usamos `-m` seguido del nombre. Aquí colocamos:

```
git commit -m "proyecto inicial"
```

Sugerimos que usen nombres explícitos e investiguen sobre las buenas prácticas. Intentaremos proporcionar material sobre estas prácticas. Incluso hay sitios que ayudan a definir esos nombres para que, al verlos, nos proporcionen información importante, como "proyecto inicial". Ya sabemos que es un proyecto inicial. Perfecto, el repositorio local está listo. Observen que el código cambió y quedó en blanco.

Definiendo ramas y conectando con el repositorio remoto

Para subir esto a la nube, debemos definir algo llamado ramas. Git se comporta como un árbol con muchas líneas o ramas, que son las diferentes versiones disponibles. Todo repositorio debe tener una rama principal. Si revisan la receta, nos indica usar `git branch main` o `master` para definir la principal. Entonces, ejecutamos:

```
git branch -m main
```

Perfecto, hemos creado nuestra rama principal. Ahora debemos enlazar lo local con lo remoto.

Conexión de repositorios locales y remotos

Para enlazar un repositorio local con uno remoto, utilizamos el comando `git remote`. Es importante destacar que podemos conectarnos de local a remoto mediante HTTPS o SSH. Nosotros optamos por trabajar con SSH, ya que, una vez que identificamos quiénes somos y registramos la huella digital de nuestra máquina, el proceso de conexión se simplifica. Usar HTTPS puede ser más complicado.

Primero, copiamos la línea correspondiente, la pegamos en Visual Studio y presionamos "Enter". Con esto, hemos establecido el enlace inicial:

```
git remote add origin git@github.com:leocastillo-aluralatam/juego_secreto.git
```

Aunque aún faltan pasos, ya hemos indicado al repositorio local que se conectará al remoto.

A continuación, debemos realizar un *push* para subir los cambios. Utilizamos el comando:

```
git push -u origin main
```

El término **origin** proviene del paso anterior, cuando creamos el **remote** y le asignamos un nombre. Es una buena práctica mantener este nombre, aunque es posible cambiarlo. También es factible tener múltiples conexiones remotas, lo cual es parte de un manejo avanzado de Git.

Generando y configurando llaves SSH

Al ejecutar el comando, puede que nos indique que no estamos autenticados. En este caso, necesitamos crear una llave SSH. Este paso está documentado y consiste en ejecutar un comando que incluye nuestro correo electrónico, el mismo que usamos al configurar **user.email**:

```
ssh-keygen -t ed25519 -C "leocastillo.aluralatam@gmail.com"
```

Al crear la llave, se nos preguntará si queremos establecer una clave para ella. Recomendamos dejarla sin clave para facilitar el acceso. Una vez creada, la llave se encuentra en la ruta especificada, generalmente en **C:\Users\[usuario]\.ssh\id_rsa.pub**. Abrimos este archivo con un editor de texto y copiamos su contenido.